Database Systems

Piya Limcharoen

Version 2026/1

Contents

Chapter 4: Distributed Architecture and Key-Value Databases 6
1.1 Chapter Overview . . . 6
1.2 Recap: Core Concepts from Chapter 3 . . . 8
1.2.1 Distributed Systems: Scale-Up vs. Scale-Out . . . 8
1.2.2 Distributed Foundations: CAP, PACELC, and BASE Recap . . . 9
1.2.3 The NoSQL Landscape: 4 Data Models at a Glance . . . 14
1.3 From Document Databases to Key-Value Databases . . . 17
1.3.1 Recap: Document Databases . . . 17
1.3.2 Introduction to Key-Value Databases . . . 17
1.4 Distributed Key-Value Architecture . . . 18
1.4.1 What is a Key-Value Store? . . . 18
1.4.2 Data Partitioning . . . 21
1.4.3 Consistent Hashing: The Circular Hash Ring . . . 24

1.4.4 Consistent Hashing: Adding a Node (Scale-Out) . . . 26
1.4.5 Consistent Hashing: Removing a Node (Scale-In / Decommission) . . . 27
1.4.6 Virtual Nodes (VNodes) for Load Balancing . . . 30
1.4.7 Replication Strategies: Leader-Follower, Leaderless, and Multi-Leader . . . 32
1.4.8 Quorum Consensus ($R + W > N$) . . . 34
1.4.9 Configuring Quorum Trade-Offs . . . 37
1.4.10 Conflict Resolution: Last-Write-Wins (LWW) vs. Vector Clocks . . . 38
1.5 Redis: High-Performance In-Memory Key-Value Architecture . . . 40
1.5.1 Overview and Design Philosophy . . . 40
1.5.2 The Single-Threaded Reactor Event Loop . . . 41
1.5.3 Core Data Structures and Memory Expiration (TTL) . . . 42
1.5.4 Caching Architecture: The Cache-Aside Pattern (Lazy Loading) . . . 44
1.6 Redis: In-Memory Key-Value Store . . . 47
1.6.1 Basic Operations . . . 47
1.6.2 Transactions in Redis . . . 48
1.6.3 Redis with Python . . . 49
1.7 Chapter Summary . . . 51
1.8 Activity/Homework 1: Key-Value Stores with Python and Redis . . . 52
1.8.1 Laboratory 1: Core Setup, Strings, Hashes, and Sets . . . 52
1.8.2 Laboratory 1 (Continued): Queues, Leaderboards, and TTL Expiration . . . 53
1.8.3 Laboratory 2: Practical Cache-Aside Caching in Python . . . 54
1.9 Activity/Homework 2: Multi-Database Web Application with Redis Caching . . . 55
1.9.1 Overview . . . 55
1.9.2 Learning Objectives . . . 55
1.9.3 System Architecture . . . 56
1.9.4 Database Design . . . 56
1.9.5 Activity Tasks . . . 58

1.9.6 Deliverables 59

Copyright Notice

Copyright © 2026 Piya Limcharoen. All rights reserved.

This lecture note was authored by Piya Limcharoen. Artificial Intelligence (AI) was utilized as a tool for language enhancement and grammatical correction; however, all intellectual property, original concepts, and the structure of this work belong exclusively to the author.

Terms of Use

This document is provided for educational purposes only and is intended for the exclusive use of students currently enrolled in this course.

The reproduction, distribution, public display, or transmission of this material, in whole or in part, in any form or by any means—electronic, mechanical, photocopying, recording, or otherwise—is strictly prohibited without the prior written permission of the author. Students may not share, upload, or distribute these notes outside of the official class environment.

Warning

Any unauthorized use of this material will be considered academic misconduct and will be subject to university disciplinary procedures. Furthermore, such an act may constitute an infringement of copyright law and is subject to legal action.

Chapter 4:
Distributed Architecture and Key-Value Databases

Chapter 4: Distributed Architecture and Key-Value Databases

1.1 Chapter Overview

In Chapter 1, we established how relational databases uphold data integrity through **ACID transactions** and manage concurrency via protocols like 2PL and MVCC on a single node.

In Chapter 2, we examined how storage engines optimize physical page access and leverage **B+ Tree Indexes** and **Buffer Pools** to manage the memory wall.

In Chapter 3, we crossed into distributed systems, exploring why web-scale applications encounter the **Distributed Scale Wall**, introducing the four NoSQL data models, and mastering **Document Modeling with MongoDB**.

In this chapter, we synthesize and review the distributed foundations from Chapter 3, and then delve deeply into the simplest, fastest, and most ubiquitous NoSQL category: **Key-Value Databases (Redis)**. We will study the distributed mechanics that make them scale horizontally, analyze the internals of in-memory caching, and build practical multi-database applications.

Learning Objectives

By the end of this chapter, you will be able to:

- **Evaluate Distributed Trade-Offs (CAP & PACELC):**
  Analyze NoSQL paradigms and assess availability, consistency, and latency trade-offs during network partitions (P) and normal operations.
- **Design Scalable Partitioning with Consistent Hashing:**
  Apply circular hash rings and **Virtual Nodes (VNodes)** to balance data distributions and minimize remapping (K/N) during cluster scaling.
- **Configure Quorum Consensus & Replication:**
  Tune Read/Write quorums ($R + W > N$) in leaderless architectures to guarantee strong consistency and evaluate sloppy quorums with hinted handoff.
- **Resolve Distributed Write Conflicts:**
  Contrast the risks of physical clock drift in **Last-Write-Wins (LWW)** with causal conflict detection using **Vector Clocks**.
- **Analyze Redis Internals & Core Data Structures:**
  Explain single-threaded throughput via the **Reactor Event Loop** and non-blocking I/O, and select optimal types (Strings, Hashes, Lists, Sets, ZSets).
- **Implement Production Caching Strategies:**
  Build the **Cache-Aside (Lazy Loading)** pattern in Python, configure **TTL expiration**, and enforce **Delete-on-Write** cache invalidation.
- **Architect Polyglot Multi-Database Systems:**
  Integrate **MySQL** (ACID), **MongoDB** (flexible documents), and **Redis** (in-memory cache) into a unified, high-performance web service.

1.2 Recap: Core Concepts from Chapter 3

1.2.1 Distributed Systems: Scale-Up vs. Scale-Out

When applications grow from gigabytes to petabytes and traffic surges to millions of concurrent requests, a single database server hits the **Distributed Scale Wall**. In Chapter 3, we analyzed the architectural shift from single-node to distributed architectures:

Dimension	**Vertical Scaling (Scale-Up)**	**Horizontal Scaling (Scale-Out)**
Mechanism	Upgrading single server CPU, RAM, NVMe	Adding clusters of commodity nodes
Transactions	Strict, simple single-node ACID[1]	Requires distributed consensus (BASE[2])
Limitations	Physical hardware ceiling, exponential cost, SPOF[3]	Network latency, partitions, partial failures
Core Patterns	Monolithic RDBMS (PostgreSQL, MySQL)	Distributed NoSQL (Cassandra, Redis, Mongo)

Table 1: Recap: Architectural Trade-Offs in System Scaling

[1] **ACID:** Atomicity, Consistency, Isolation, Durability
[2] **BASE:** Basically Available, Soft state, Eventual consistency
[3] **SPOF:** Single Point of Failure

1.2.2 Distributed Foundations: CAP, PACELC, and BASE Recap

When distributing data across clusters of commodity servers, systems must navigate physical network realities:

- **The CAP Theorem:** Network partitions (P) are an unavoidable physical reality in distributed networks. When a partition occurs, systems must make an architectural trade-off:
  - **CP (Consistency + Partition Tolerance):**
    Refuse or block writes to preserve linearizable correctness.
    In short: keep data correct, even if some requests must wait or fail.
  - **AP (Availability + Partition Tolerance):**
    Keep serving writes on all nodes, accepting temporary divergence that is reconciled later.
    In short: keep the system running and respond quickly, then synchronize data later.

- **The PACELC Theorem:**
  Extends CAP to normal operations (when all nodes can communicate): when there is no network partition, the system still faces a trade-off between responding quickly (Latency) and keeping data consistent (Consistency).

  If Partition (**P**) → trade **A** vs. **C**; **E**lse → trade **Latency (L)** vs. **Consistency (C)**.

- **ACID vs. BASE:** Relational databases prioritize immediate, strong consistency (ACID). Distributed NoSQL systems embrace **BASE**: **B**asically **A**vailable, **S**oft state (asynchronous propagation), and **E**ventual consistency.

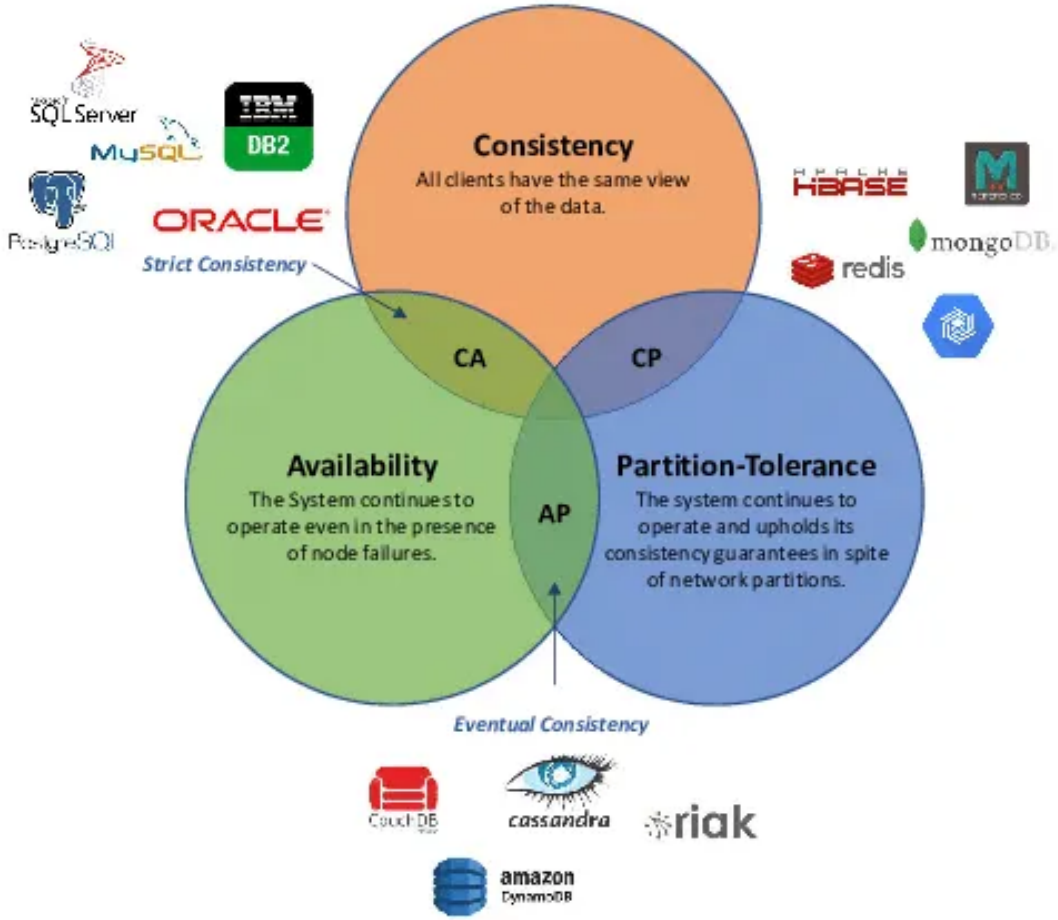


Figure 1: The CAP Theorem: Consistency, Availability, and Partition Tolerance

Source: Logiwa Tech - What is the CAP Theorem? Understanding Consistency, Availability, and Partition Tolerance

1.2.2.1 CP vs. AP Systems

Aspect	**CP Systems**	**AP Systems**
Design Goal	Correctness first	Availability first
Behavior on Partition	Stops or rejects operations	Continues operating
Consistency Model	Strong consistency	Eventual consistency
Failure Handling	Fail-safe (stop to stay correct)	Fail-soft (continue with risk)
User Impact	Possible downtime or errors	Always responsive, temporary inconsistencies
Typical Domains	Finance, healthcare, coordination services	Social platforms, IoT, real-time analytics
Example Systems	ZooKeeper, HBase, MongoDB (CP modes)	Cassandra, DynamoDB, Riak

1.2.2.2 ACID vs. BASE Comparison

Feature	**ACID (RDBMS)**	**BASE (NoSQL)**
Primary Goal	Strong Consistency	High Availability
Transaction Scope	Single Node / Strict	Distributed / Flexible
Data Integrity	Immediate	Eventual
Best Use Case	Financial Systems, Inventory	Social Media, Real-time Analytics

Table 2: ACID vs. BASE Comparison

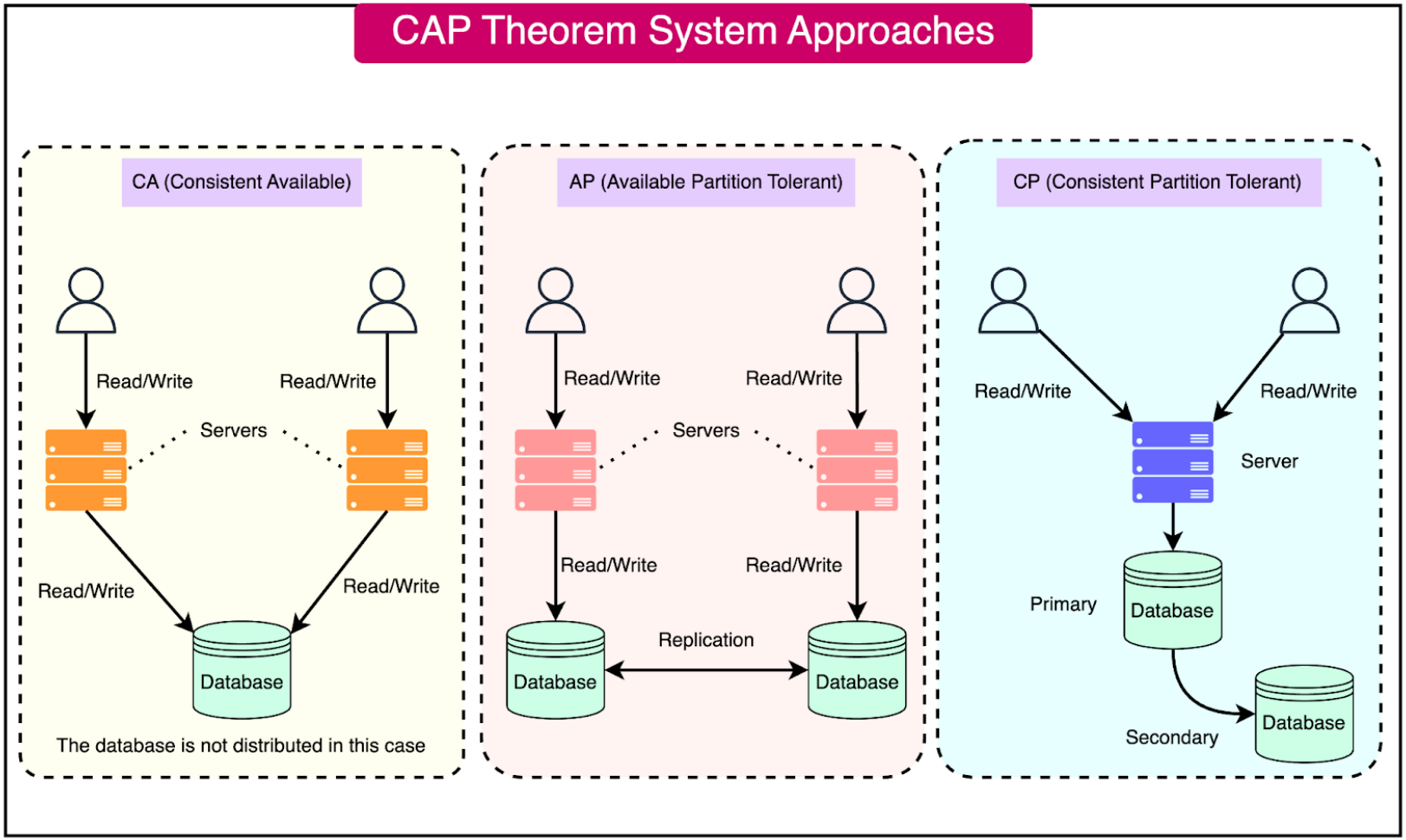


Figure 2: CAP, PACELC, ACID, and BASE Concepts

Source: ByteByteGo - Essential Concepts

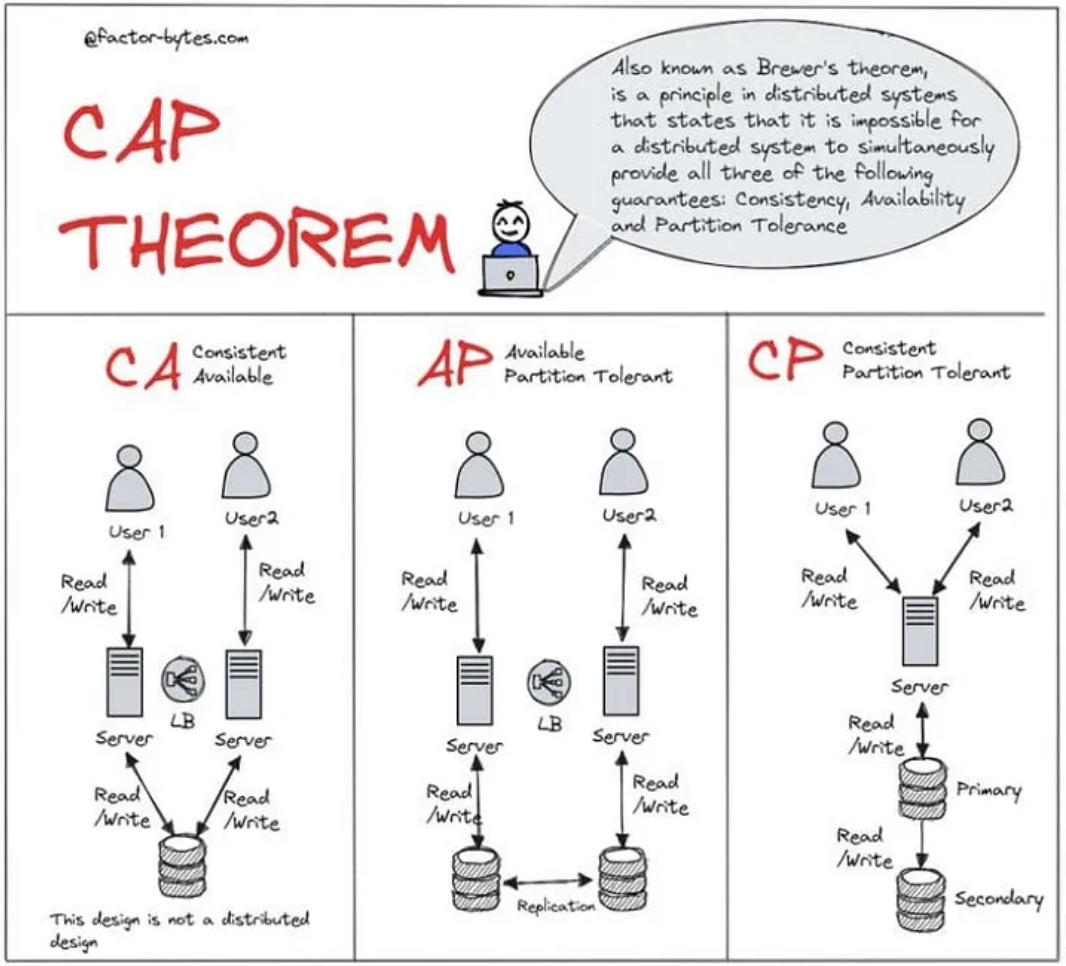


Figure 3: CAP Theorem Visualization

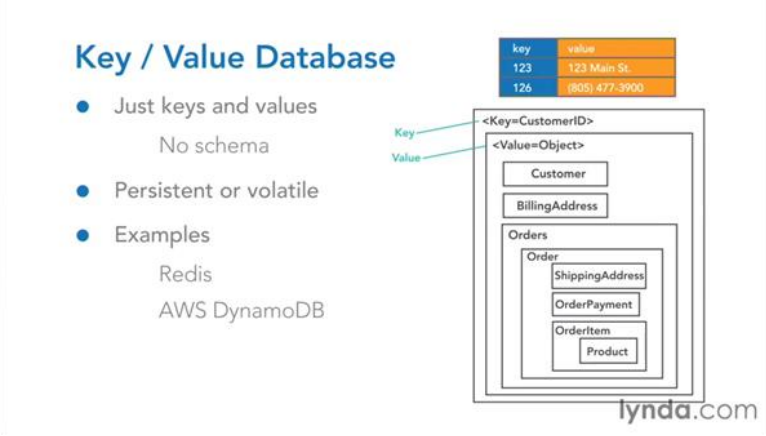
Source: Medium - CAP Theorem Visualization

1.2.3 The NoSQL Landscape: 4 Data Models at a Glance

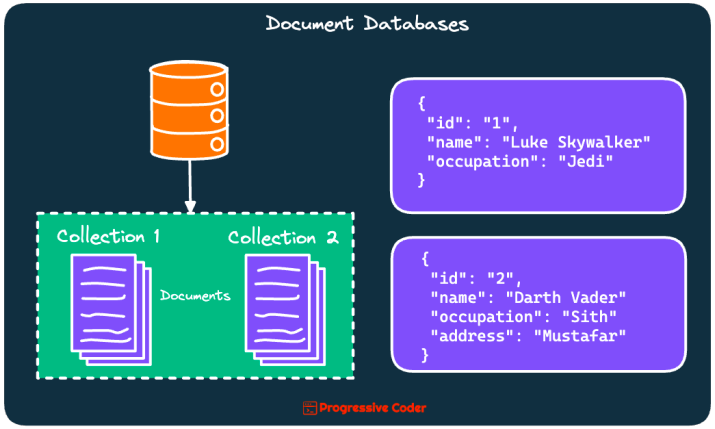
In Chapter 3, we examined the motivation behind NoSQL systems: overcoming the relational scale wall to handle high write velocity, massive data volumes, and flexible schemas. NoSQL databases are organized by data model into four primary categories:

Model	**Data Structure & Access**	**Primary Domain & Course Coverage**
Key-Value	Unique Key maps to opaque Value ($O(1)$)	Caching, sessions, real-time counters (**Focus of Chapter 4**)
Document	Hierarchical JSON/BSON documents	Catalogs, CMS, semi-structured profiles (**Covered in Chapter 3: MongoDB**)
Wide-Column	Sparse multidimensional table	Massive time-series, IoT telemetry (**Covered in Chapter 5: Cassandra**)
Graph	Explicit Nodes and Edges	Social networks, fraud detection (**Covered in Chapter 5: Neo4j**)

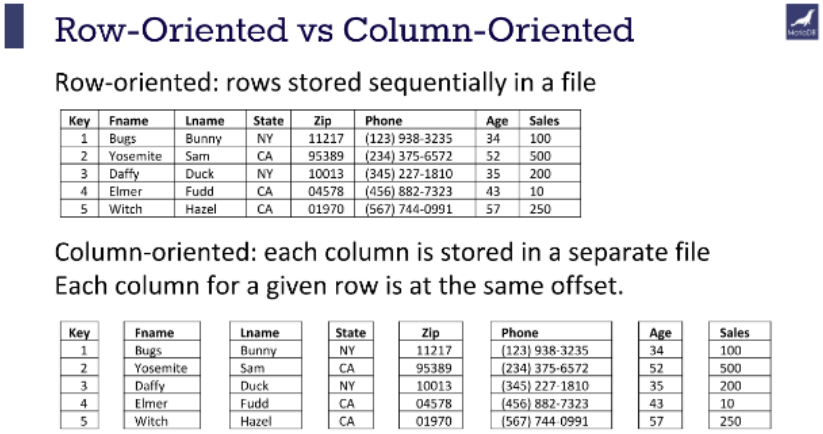
Table 3: Recap: The Four Primary NoSQL Database Models



(a) Key-Value Database



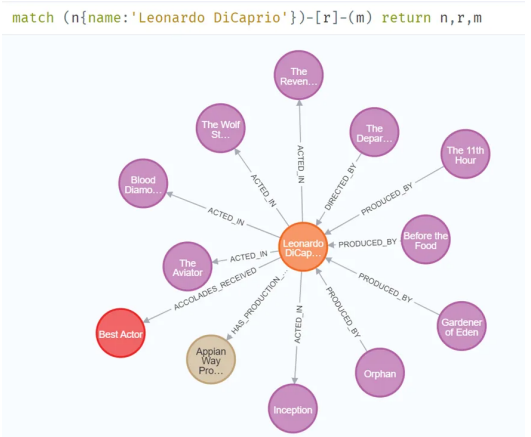
(b) Document Database



Key	Fname	Lname	State	Zip	Phone	Age	Sales
1	Bugs	Bunny	NY	11217	(123) 938-3235	34	100
2	Yosemite	Sam	CA	95389	(234) 375-6572	52	500
3	Daffy	Duck	NY	10013	(345) 227-1810	35	200
4	Elmer	Fudd	CA	04578	(456) 882-7323	43	10
5	Witch	Hazel	CA	01970	(567) 744-0991	57	250

Key	Fname	Lname	State	Zip	Phone	Age	Sales
1	Bugs	Bunny	NY	11217	(123) 938-3235	34	100
2	Yosemite	Sam	CA	95389	(234) 375-6572	52	500
3	Daffy	Duck	NY	10013	(345) 227-1810	35	200
4	Elmer	Fudd	CA	04578	(456) 882-7323	43	10
5	Witch	Hazel	CA	01970	(567) 744-0991	57	250

(c) Row vs. Column-Oriented Storage



(d) Graph Database

Figure 4: Architectural Structures of the Four Primary NoSQL Models: (a) Key-Value Store, (b) Document Database, (c) Column-Oriented Storage, and (d) Graph Database with Explicit Relationships.

Sources: (a) The Customize Windows, (b) ByteByteGo, (c) MariaDB, (d) ignio

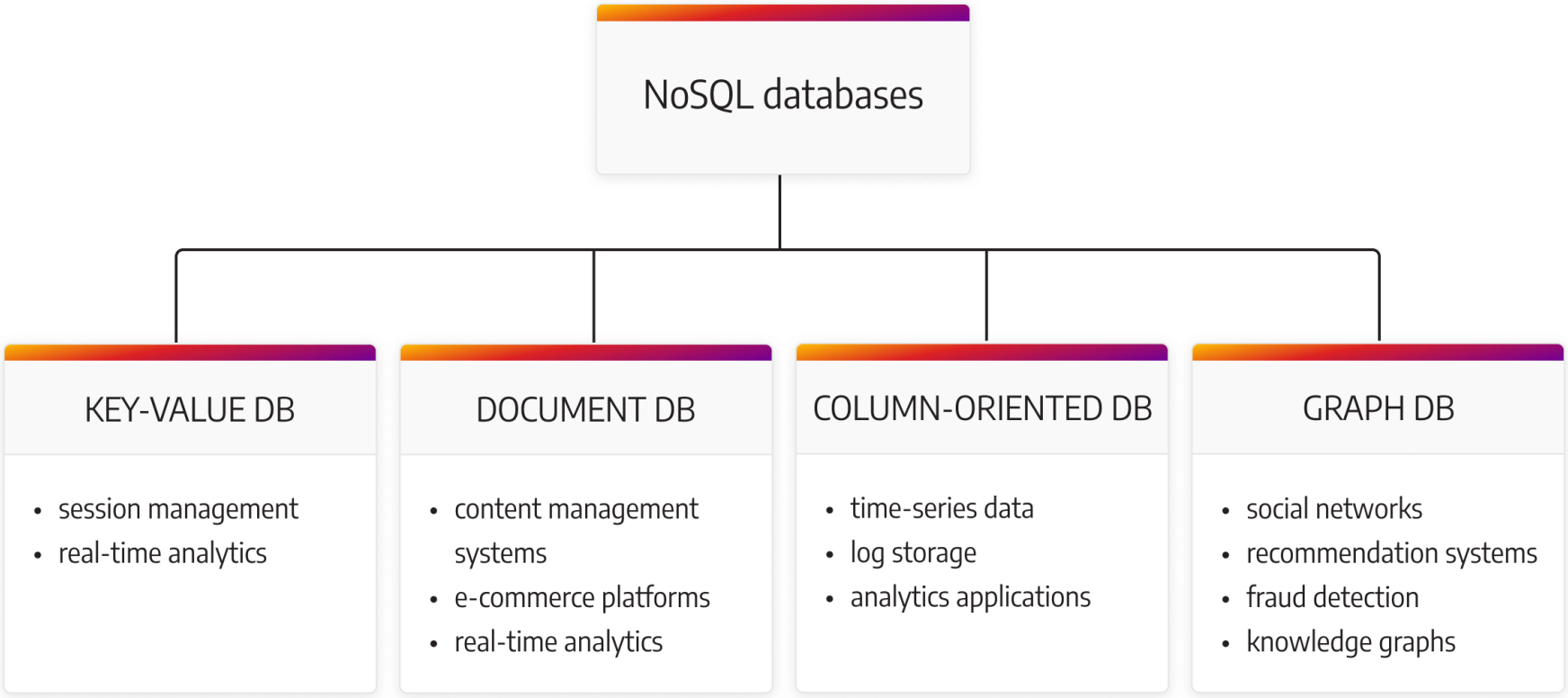


Figure 5: Types of NoSQL Databases

Source: Memgraph - When to use a NoSQL database

1.3 From Document Databases to Key-Value Databases

1.3.1 Recap: Document Databases

- Store data as documents using formats such as JSON (JavaScript Object Notation), organized as key-value pairs and nested objects.
- Support embedding and referencing to represent relationships between documents.
- MongoDB provides CRUD operations and aggregation for querying and processing documents.
- MongoDB stores documents internally in BSON (Binary JSON), a binary format that supports additional data types.
- XML (Extensible Markup Language) represents structured data using nested elements and tags. Relational databases such as Oracle Database, Microsoft SQL Server, and PostgreSQL also support XML storage and querying.

1.3.2 Introduction to Key-Value Databases

- Store data as key-value pairs, where each key identifies its associated value.
- Provide simple and fast access to data using keys.
- Redis is an in-memory key-value database commonly used for caching and fast data access.

1.4 Distributed Key-Value Architecture

1.4.1 What is a Key-Value Store?

At its architectural core, a **Key-Value (KV) Store** is a distributed hash table:

- **Interface:** Provides a minimal, highly optimized API:

  `get(key)` | `put(key, value)` | `delete(key)`

- **Opaque Values:** From the database engine's perspective, the value is an unstructured byte array (blob). The engine does not inspect, parse, or index inner fields of the value.

- **Storage Tiers:**
  - **In-Memory KV Stores (e.g., Redis, Memcached):**
    Data resides entirely in RAM for sub-millisecond throughput. Persistence to disk is optional/asynchronous.
  - **Persistent Disk-Backed KV Stores (e.g., Amazon DynamoDB, RocksDB):**
    Data is permanently persisted to disk, using RAM for caching and LSM-tree structures.

Key	Value
123	123 Main St.
126	(805) 477-3900

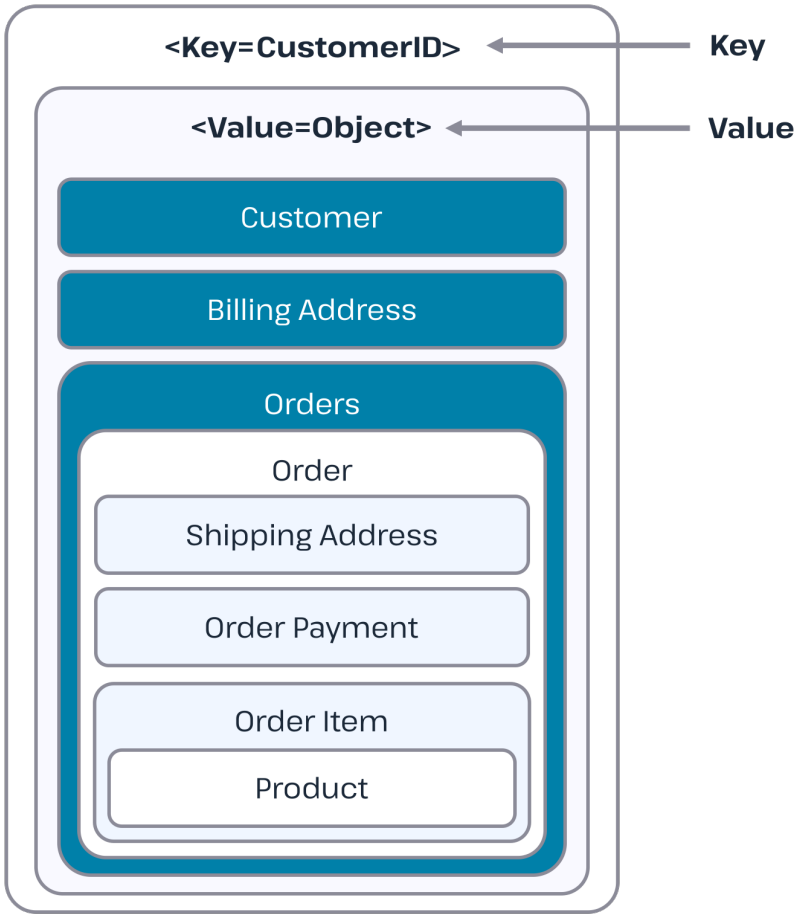


Figure 6: Key-Value Store Conceptual Architecture

Source: Hazelcast - What is a Key-Value Store?

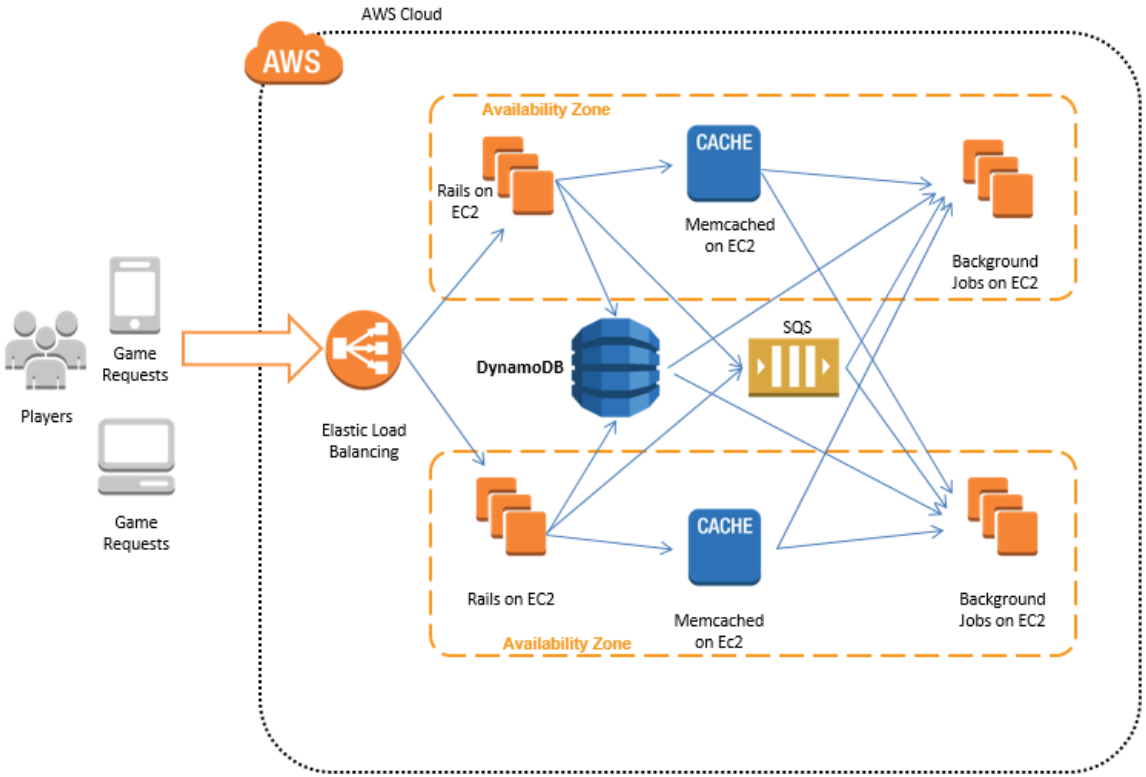


Figure 7: Amazon DynamoDB Architecture: Distributed Partitions and Storage Nodes

Source: ScyllaDB - Introduction to DynamoDB

1.4.2 Data Partitioning

To store billions of keys across N physical cluster nodes, we must partition the dataset:

1. **Range Partitioning:**
   Keys are grouped alphabetically or numerically (e.g., Node 1 stores [A–F], Node 2 stores [G–M]).

   - **Limitations:**
     Leads to catastrophic **Single Point of Failure (SPOF)** when access is skewed (e.g., all users accessing keys starting with "S").

2. **Naive Modulo Hash Partitioning:**
   Keys are assigned via a uniform hash function:

   $$\text{Node Index} = \text{hash(key)} \bmod N, \text{ where } N \text{ is the number of nodes.}$$

   - **Limitations:**
     The simple modulo hash function causes a **Resizing Disaster**.

     When the number of nodes increases from N to $N + 1$ or decreases from N to $N - 1$, the modulo calculation changes. As a result, most keys may be assigned to different nodes, requiring large amounts of data to be moved across the network and potentially slowing down the entire cluster.

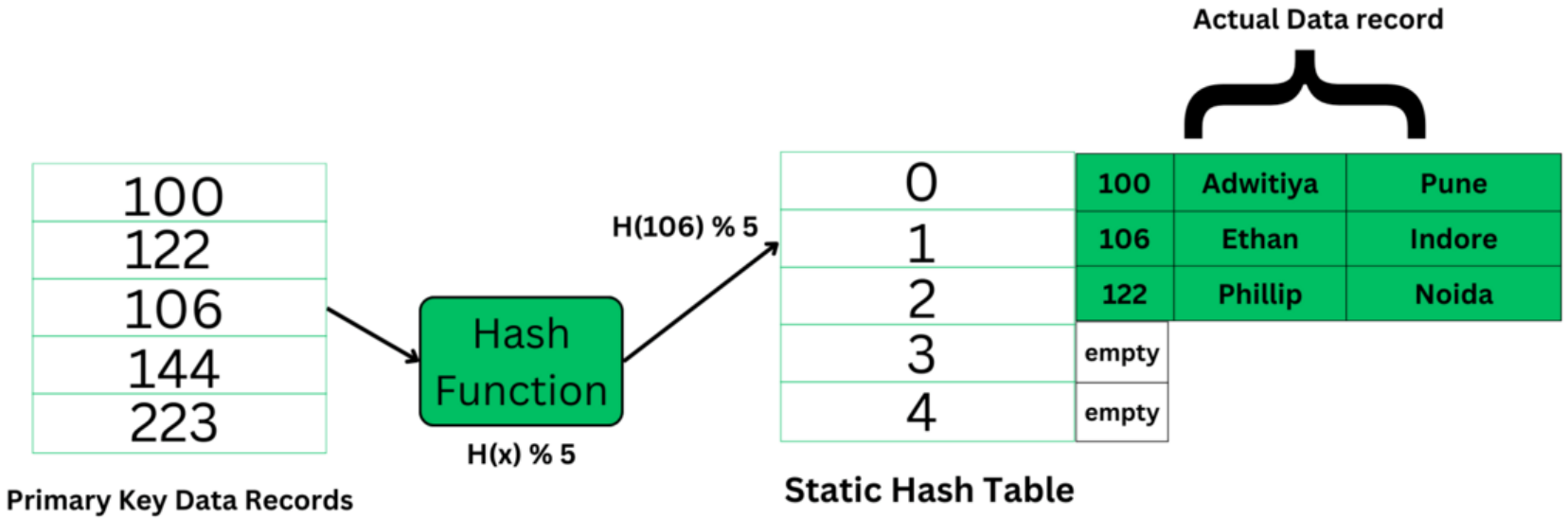


Figure 8: Traditional Hashing Algorithm in DBMS: Mapping Keys to Fixed Buckets via Modulo ($H(k) \pmod N$)

Source: GeeksforGeeks - Hashing in DBMS

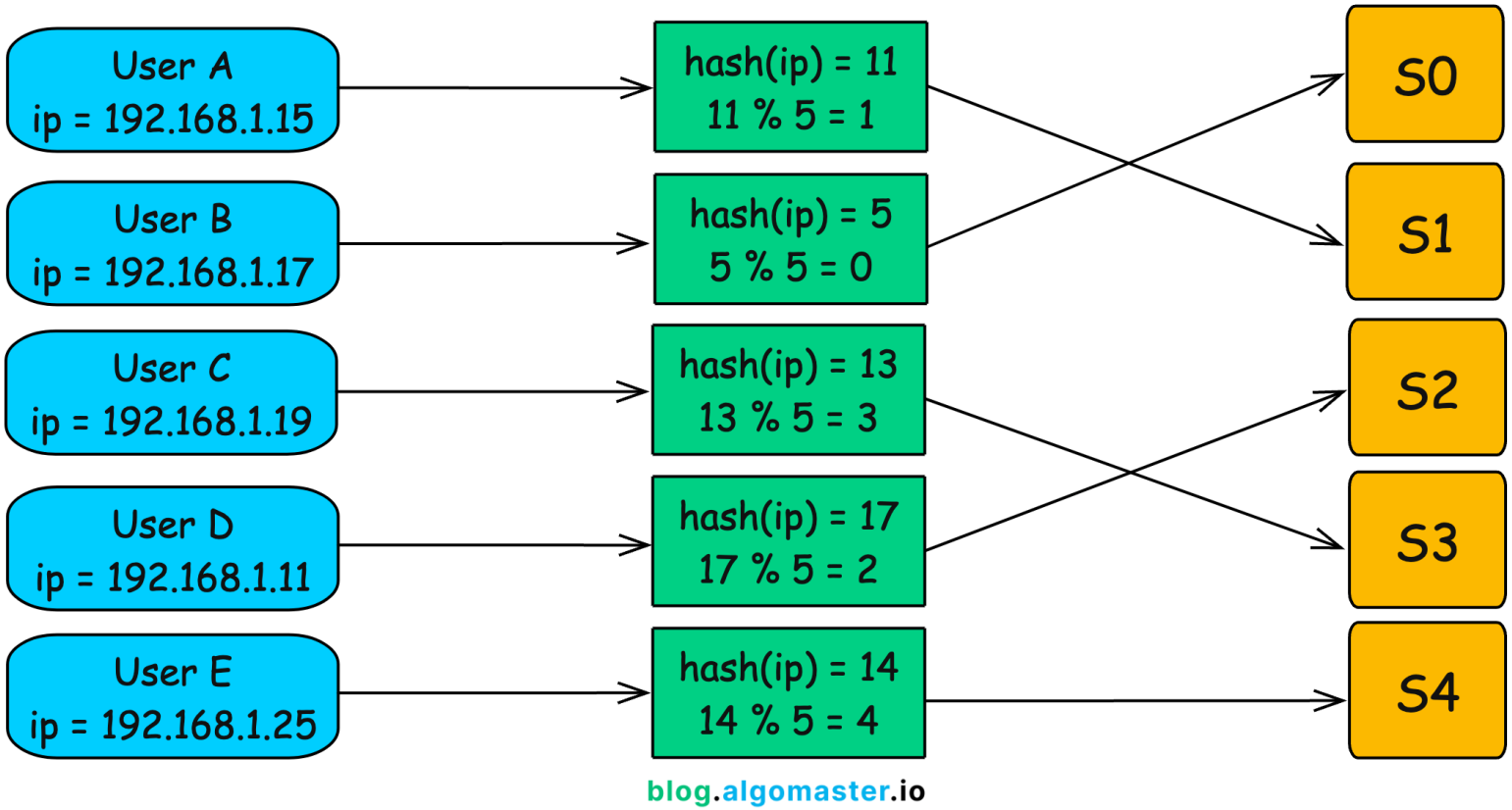


Figure 9: Naive Modulo Hashing: Mapping Client Requests to Servers via hash(key) mod N

Source: Algomaster - Consistent Hashing Explained

1.4.3 Consistent Hashing: The Circular Hash Ring

To avoid the resizing disaster of naive hashing, **Consistent Hashing** maps both nodes and keys to a fixed circular ring:

$$\text{Position} = \text{hash(item)} \bmod M$$

where M is a **fixed constant** (e.g., $M = 2^{32}$ in DynamoDB, or $M = 256$ in an 8-bit hash space) that **never changes** when the cluster scales.

- **Node Positioning:** Each node is assigned positions on the hash ring using a hash function. **Virtual nodes (vnodes)** help distribute keys more evenly across physical nodes.
- **Clockwise Routing Rule:**
  A key is stored on the first node reached moving clockwise along the ring (Node Position $\geq$ Key Position).
- **Example Equation (matching Figure 10 with $M = 256$):**
  - Nodes placed at: Node A = 39, Node C = 136, Node B = 170.
  - $\text{hash}(\text{key}_1) \bmod 256 = \mathbf{64} \implies$ routes clockwise to **Node C** ($136 \geq 64$).
  - $\text{hash}(\text{key}_2) \bmod 256 = \mathbf{200} \implies$ wraps past $255 \to 0$, routes to **Node A** (39).
- **Minimal Data Movement:**
  Adding/removing a node moves only K/N keys on average, leaving all other node partitions completely unaffected.

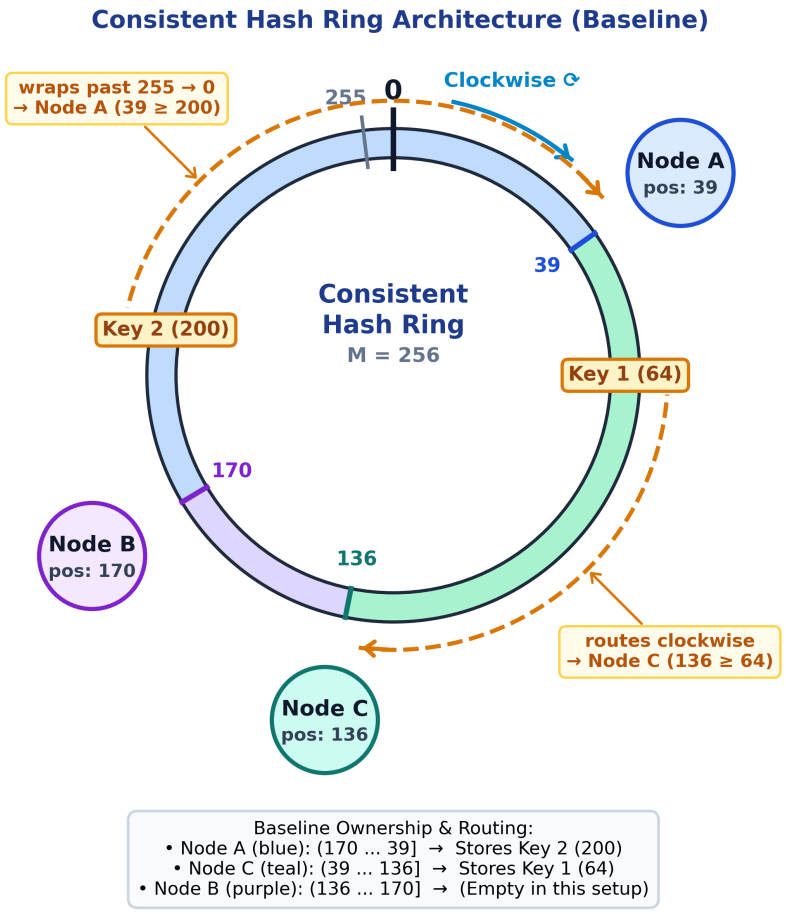
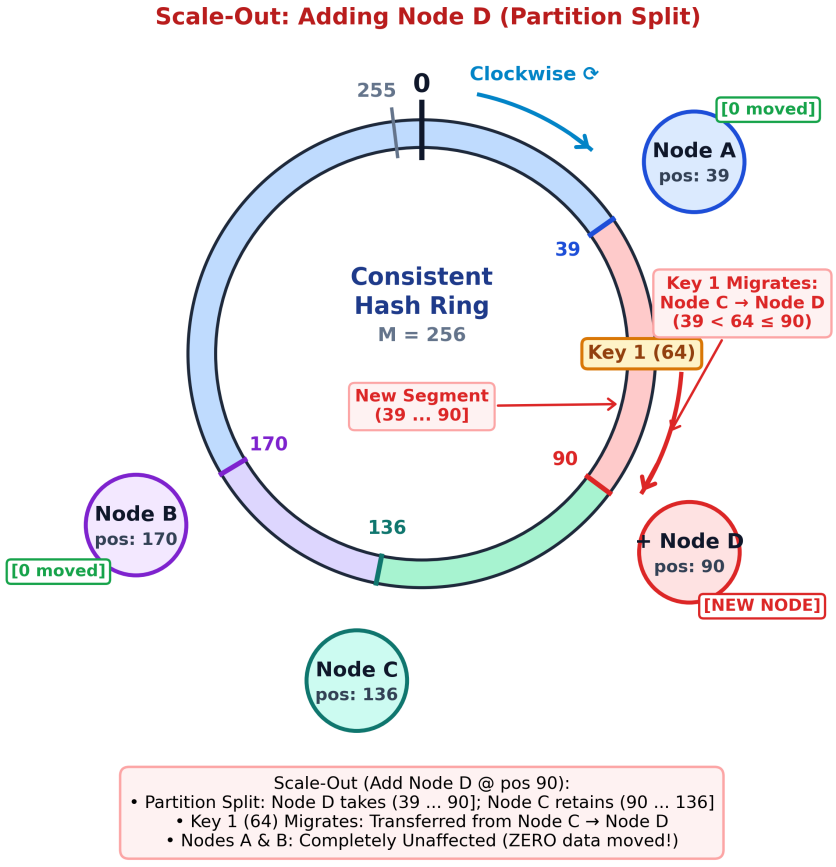


Figure 10: Consistent Hash Ring Architecture: Baseline Node Allocations and Clockwise Routing ($M = 256$)

1.4.4 Consistent Hashing: Adding a Node (Scale-Out)

Suppose the cluster scales out by adding **Node D** at position **90** (hash(Node D) mod 256 = 90):

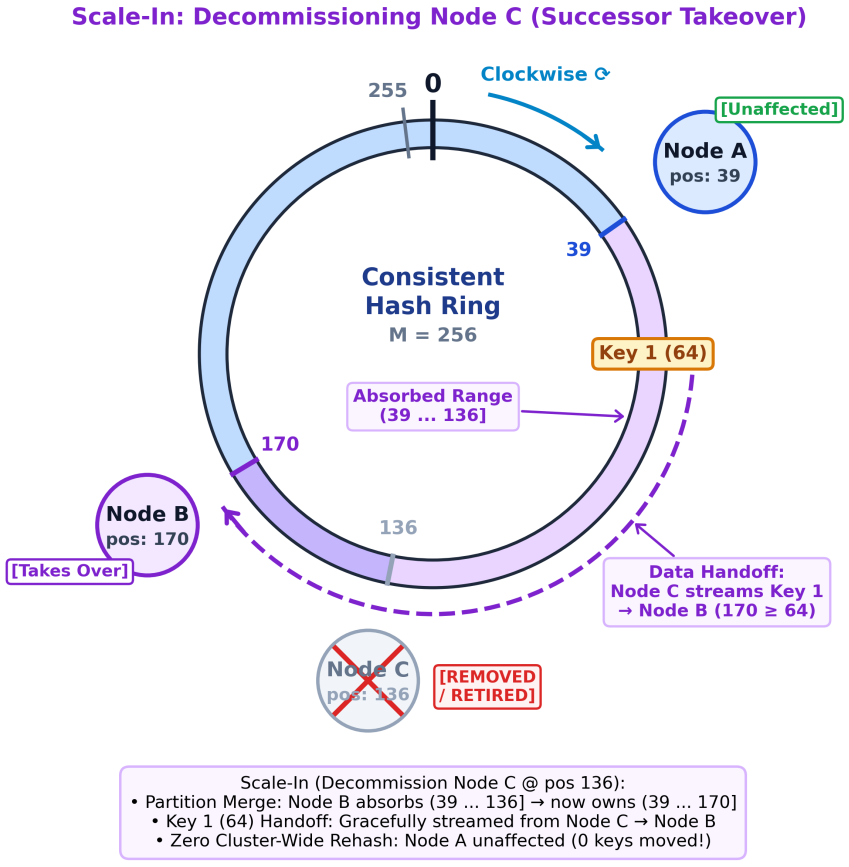
- **Split Single Partition Only:**
  - Previously, Node C owned the partition (39 . . . 136].
  - Node D is inserted between Node A (39) and Node C (136).
  - **Partition Split:**
    Node D takes (39 . . . 90]; Node C keeps (90 . . . 136].
- **Targeted Key Migration:**
  - hash(Key 1) = 64: since $39 < 64 \leq 90$,
    Key 1 moves from **Node C** → **Node D**.
  - hash(Key 2) = 200: remains on **Node A** (untouched).
- **Zero Cluster-Wide Rehash:**
  - **Node A** and **Node B** move **ZERO** keys.
  - Only $\approx \frac{K}{N+1}$ keys move when a node is added, compared with approximately $\frac{N}{N+1}K$ keys under naive modulo hashing, reducing data transfer and rehashing computation.



1.4.5 Consistent Hashing: Removing a Node (Scale-In / Decommission)

Suppose the cluster scales in by decommissioning **Node C** (at position 136):

- **Successor Partition Merge:**
  - When **Node C** is decommissioned, its token (136) is removed from the ring.
  - The next clockwise node, **Node B** (170), absorbs Node C's segment $(39 \ldots 136]$.
  - **Expanded Partition:** Node B now owns the combined range $(39 \ldots 170]$.
- **Graceful Data Handoff:**
  - Before taking itself offline, **Node C** streams its stored keys (Key 1) directly to successor **Node B**.
  - $\text{hash}(\text{Key 1}) = 64 \implies$ subsequent requests route seamlessly to **Node B** $(170 \geq 64)$.
- **Zero Cluster-Wide Rehash:**
  - **Node A** is completely unaffected (zero data moved).
  - Only Node C's keys are migrated, leaving the rest of the cluster untouched.



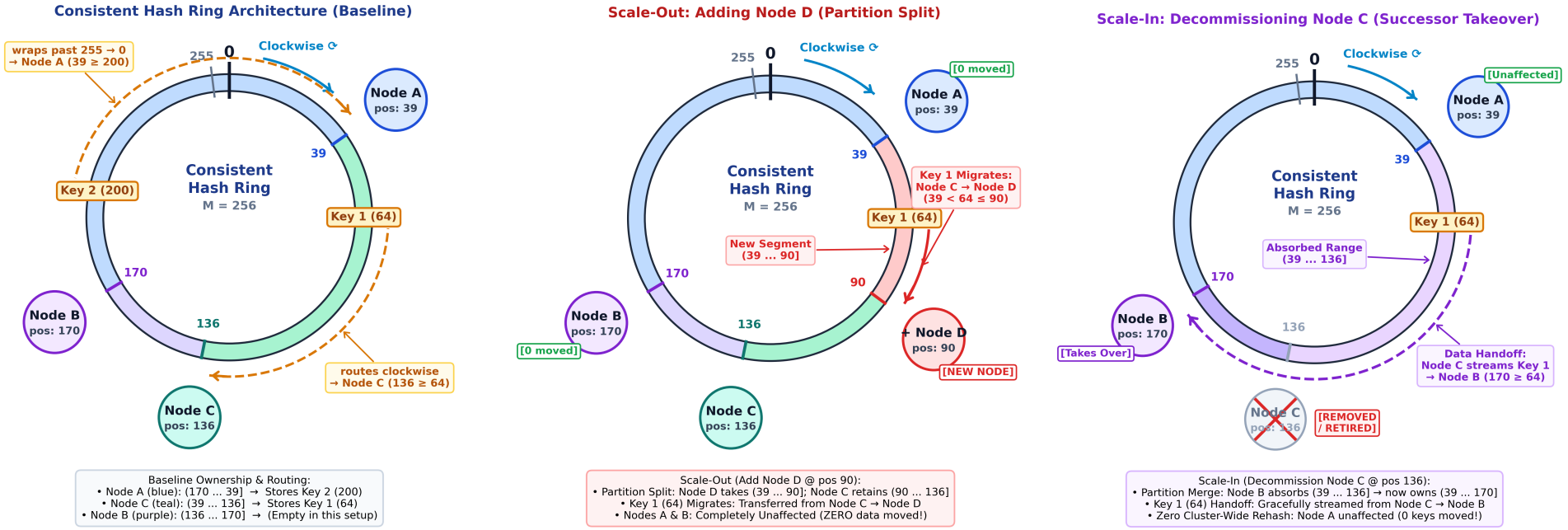
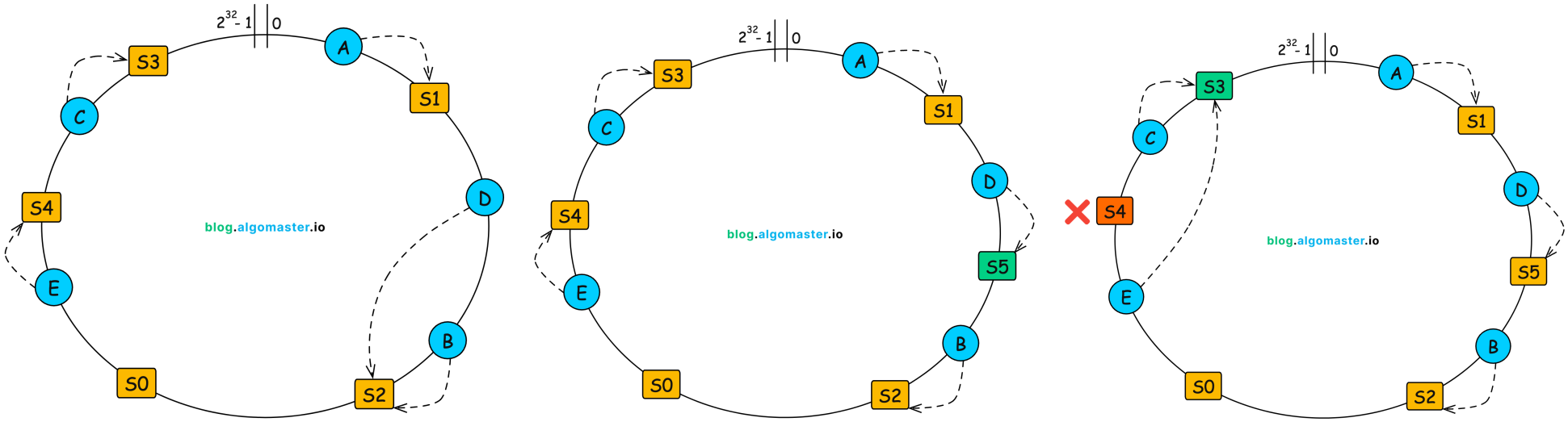


Figure 11: Consistent Hash Ring Lifecycle: Comprehensive Summary of Baseline Routing (a), Scale-Out (b), and Scale-In (c)



(a) Baseline Ring (2^{32} Key Space) (b) Adding Server S5 (Scale-Out) (c) Removing Server S4 (Scale-In)

Figure 12: Consistent Hashing Step-by-Step Example (2^{32} Hash Space): (a) Initial Server & Key Ring Mapping, (b) Scale-Out Adding Server S5, and (c) Scale-In Removing Server S4

Source: Algomaster - Consistent Hashing Explained

1.4.6 Virtual Nodes (VNodes) for Load Balancing

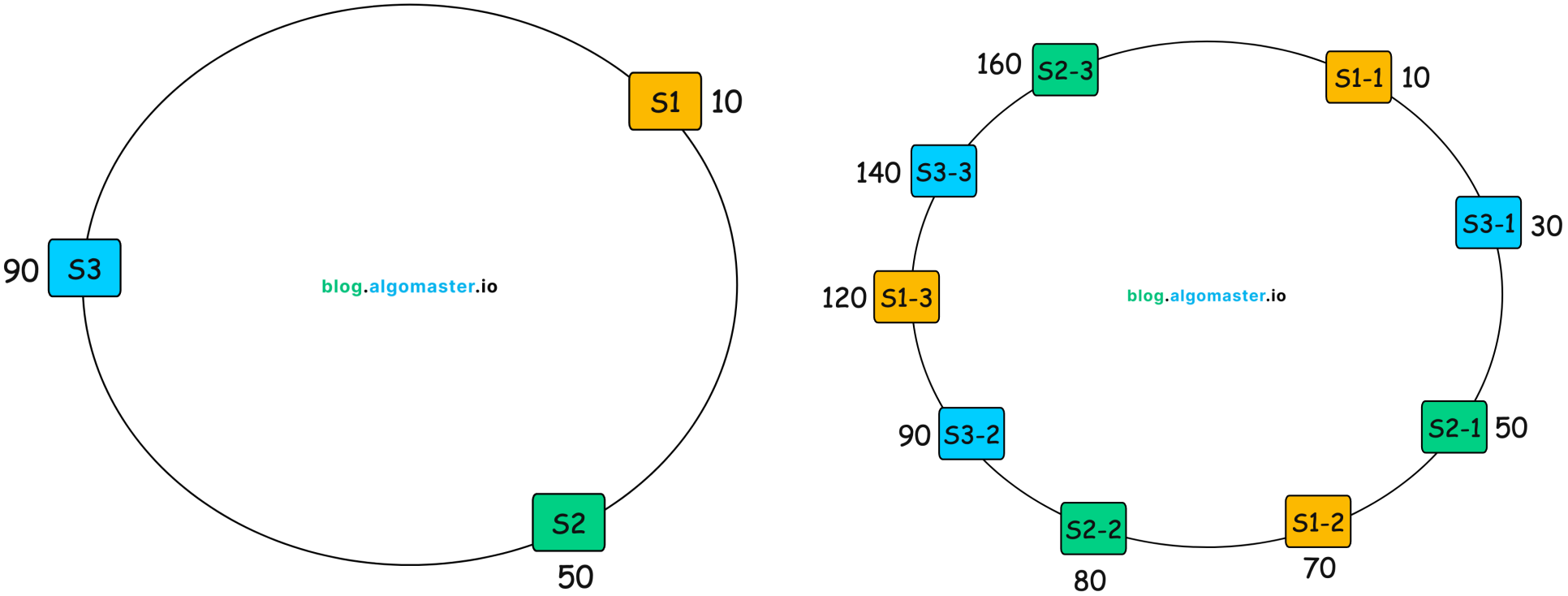
Instead of assigning one position per server, each physical server is assigned multiple positions (virtual nodes) on the hash ring. A standard Consistent Hash ring has two vulnerabilities:

1. **Non-Uniform Distribution (Hotspots):**
   With only a few physical servers, node positions may cluster on the hash ring, causing one server to own a much larger section of the ring and receive many more keys than others.
2. **Heterogeneous Hardware:**
   Powerful 128-core servers receive the same ring segment as smaller 16-core servers.

Solution: Virtual Nodes (VNodes)

Instead of mapping each physical server to a single token on the ring, each physical node is assigned V distinct **Virtual Nodes** (e.g., $V = 256$ tokens across the ring):

- Tokens are interleaved uniformly around the circle.
- If physical Node A fails, its workload is automatically dispersed across **all remaining physical nodes**, rather than overwhelming its single clockwise neighbor.
- Faster servers can be allocated a higher virtual token count (e.g., 512 VNodes vs. 128 VNodes).



(a) Without VNodes (Physical Nodes Only: S1, S2, S3)

(b) With Virtual Nodes (3 Tokens per Server Interleaved)

Figure 13: Virtual Nodes (VNodes) for Consistent Hashing: (a) A ring with only physical servers creates uneven partition boundaries and hotspot vulnerability, whereas (b) assigning multiple virtual tokens per physical node (S_i-1, S_i-2, S_i-3) ensures uniform load distribution.

Source: Algomaster - Consistent Hashing Explained

1.4.7 Replication Strategies: Leader-Follower, Leaderless, and Multi-Leader

1. **Leader-Follower Replication:**
    - A leader handles writes and replicates data to followers.
    - *Trade-off:* Simple coordination, but the leader can become a bottleneck.
    - Examples: Redis and MongoDB.

2. **Leaderless Replication:**
    - Any node can coordinate reads and writes.
    - Each write is sent to N replica nodes selected from the nodes responsible for storing the data.
    - *Trade-off:* High availability, but requires conflict resolution.
    - Examples: Amazon DynamoDB and Apache Cassandra.

3. **Multi-Leader Replication:**
    - Multiple leaders accept writes and synchronize data across regions or data centers.
    - *Trade-off:* Low write latency across regions, but concurrent writes may conflict.
    - Examples: Amazon DynamoDB Global Tables and CouchDB.

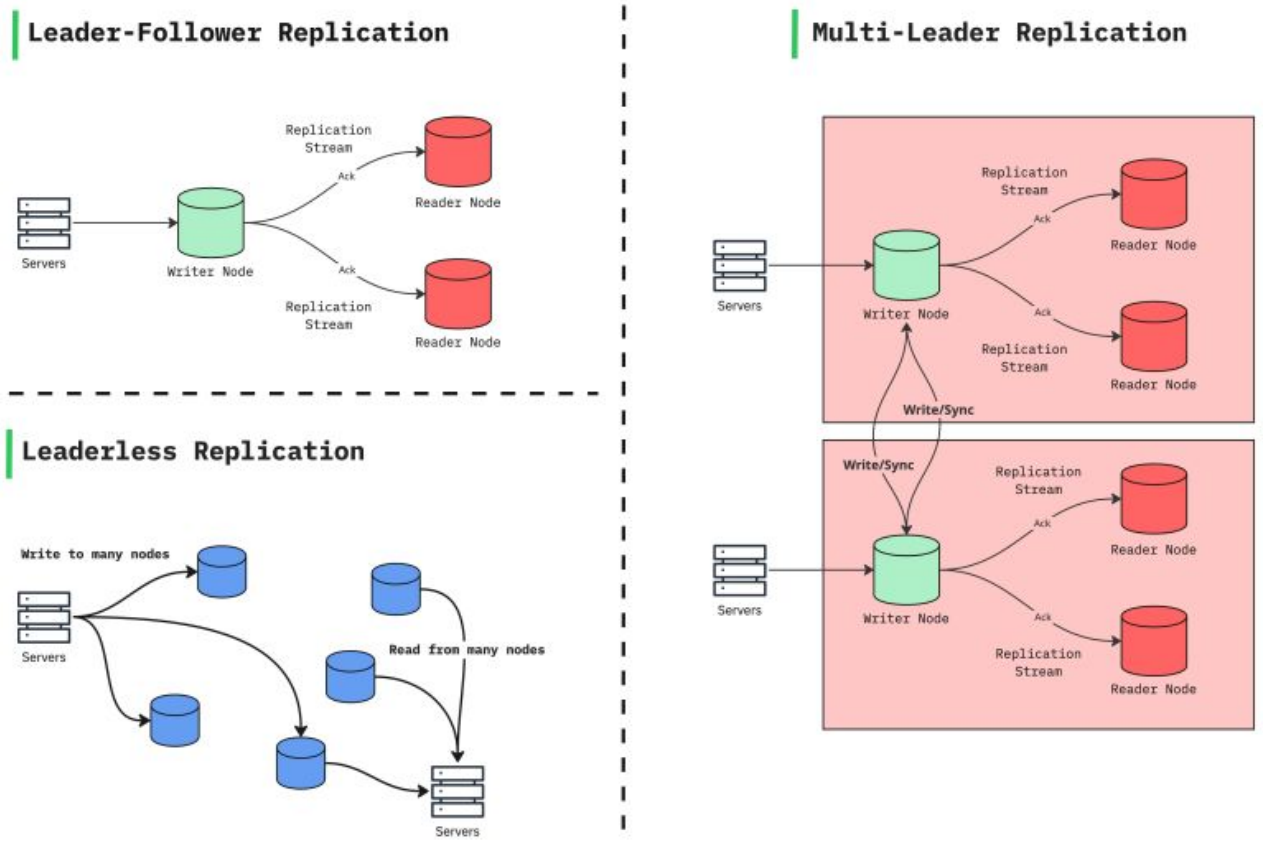


Figure 14: Database Replication Architectures: Leader-Follower, Leaderless, and Multi-Leader Replication

Source: Joud Awad - Database Replication: What Every Engineer Needs to Know

1.4.8 Quorum Consensus ($R + W > N$)

In Chapter 3 (*Replication and Agreement Among Replicas*), we introduced the concept of a **quorum** as the minimum number of replica responses required to balance availability and consistency.

In a leaderless distributed system with replication factor N, how do we configure these quorum parameters to guarantee that clients read the latest data without centralized locking?

We configure **Quorum Consensus** based on the application's goals, workload, and availability requirements using three parameters:

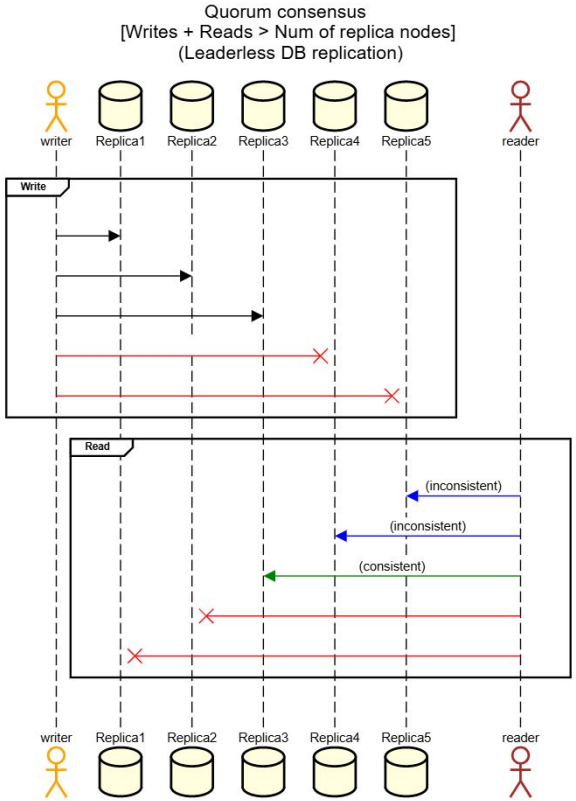
- N = Number of replicas that store a copy of the key (e.g., $N = 3$).
- W = Minimum number of replicas that must acknowledge a **write** before reporting success.
- R = Minimum number of replicas that must respond to a **read** query.

Configuration for Consistency (Trade-off Between Availability and Consistency)

Improving consistency when:

$$R + W > N$$

The read and write quorums must overlap. This ensures that at least one replica in the read quorum received the write. If the system correctly handles versions and concurrent writes, this helps reads return the latest value.



Quorum Consistency Mechanics $(N = 5, W = 3, R = 3)$

- **1. Write Phase ($W = 3$):**
  The writer commits data to Replicas 1, 2, and 3 (black arrows). Replicas 4 and 5 fail or lag (red $\times$), but the write succeeds because $W = 3$ is satisfied.
- **2. Read Phase ($R = 3$):**
  The reader queries Replicas 3, 4, and 5. Although Replicas 4 and 5 return stale (not latest) data (blue arrows), **Replica 3** returns the fresh write (green arrow).
- **3. Overlap Guarantee ($W + R > N$):**
  Since $3 + 3 = 6 > 5$, the write set $\{1, 2, 3\}$ and read set $\{3, 4, 5\}$ are mathematically guaranteed to overlap on **Replica 3**, ensuring the latest update is always detected.

Source: Kishore Sowdi - Quorum-Based Consistency ($W + R > N$) Explained

Figure 15: Quorum-Based Consistency ($W + R > N$) in Leaderless Database Replication

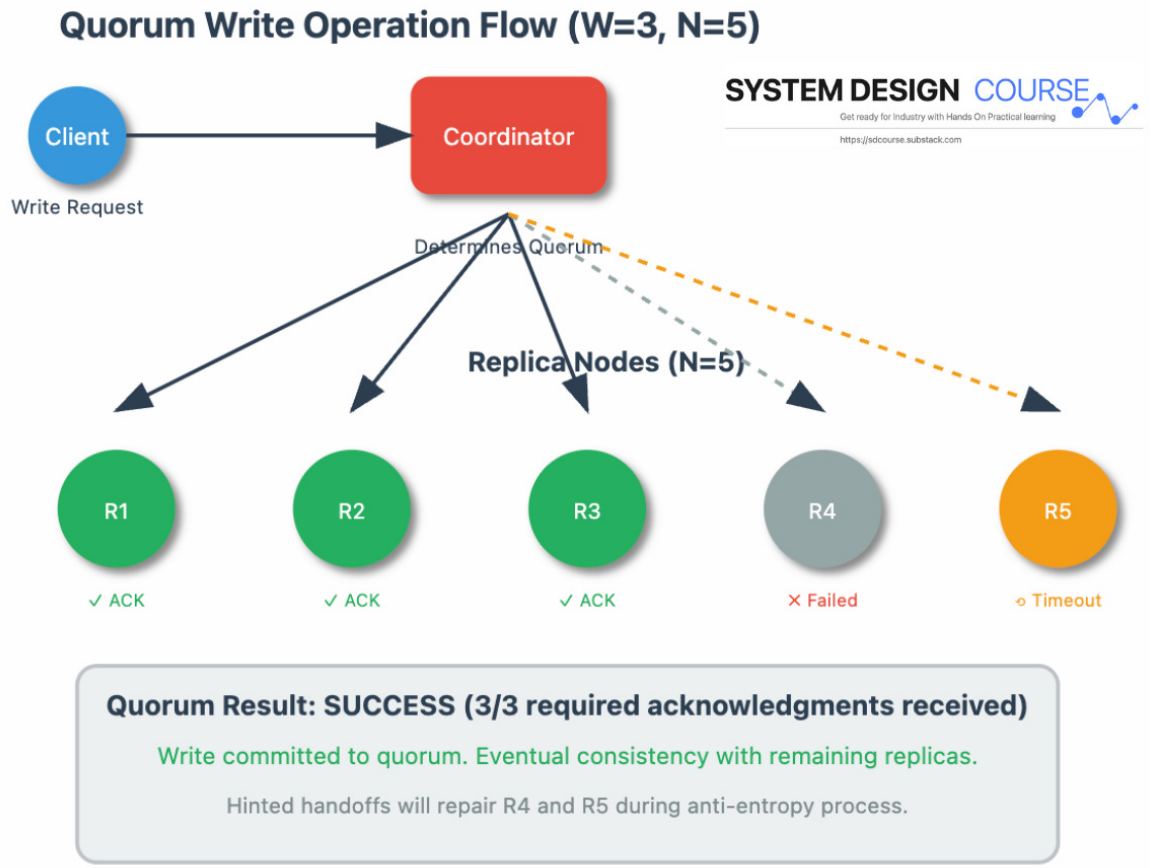


Figure 16: Quorum Write Operation Flow ($W = 3, N = 5$): The Coordinator successfully commits the write upon receiving $W = 3$ acknowledgments from replicas R1, R2, and R3; remaining lagging/failed nodes (R4, R5) are later synchronized via hinted handoffs and anti-entropy.

Source: System Design Review - Quorum-Based Systems: Ensuring Data Consistency in Distributed Databases

1.4.9 Configuring Quorum Trade-Offs

For N replicas, a write is sent to the replicas, but only W replicas must acknowledge it for the write to succeed.

Similarly, R specifies how many replicas must respond to a read. Read responses may contain stale (out-of-date) or conflicting values, depending on replica synchronization and conflict resolution.

The following table shows the trade-offs for $N = 3$:

Configuration	**Parameters**	**Advantages**	**Trade-offs**	**Overlap Guarantee**
Fast Writes	$W = 1,\ R = 3$	Writes need only one acknowledgment.	Reads require all three replicas to respond and may fail if any replica is unavailable.	Yes, since $R + W > N$.
Fast Reads	$W = 3,\ R = 1$	Reads need only one replica to respond.	Writes require all three acknowledgments and may fail if any replica is unavailable.	Yes, since $R + W > N$.
Balanced Quorum	$W = 2,\ R = 2$	Reads and writes can succeed with two available replicas.	Requires at least two available replicas for each operation.	Yes, since $R + W > N$.
Weak Quorum	$W = 1,\ R = 1$	Reads and writes need only one acknowledgment or response.	Read and write quorums may not overlap, increasing the risk of stale (out-of-date) reads.	No, since $R + W \leq N$.

Table 4: Quorum Configurations and Their Trade-Offs ($N = 3$)

1.4.10 Conflict Resolution: Last-Write-Wins (LWW) vs. Vector Clocks

In leaderless systems, concurrent writes to the same key can create conflicts.

- **Last-Write-Wins (LWW)**
  - Each write has a timestamp. The write with the latest timestamp is kept.
  - *Limitation:* Clock differences between servers may cause a newer update to be overwritten, resulting in silent data loss.
- **Vector Clocks (Version Vectors)**
  - Track the order of updates using logical counters for each node, rather than physical timestamps.
  - Detect concurrent writes when neither version is newer than the other.
  - Both versions are kept for the application to resolve, such as merging items in a shopping cart.

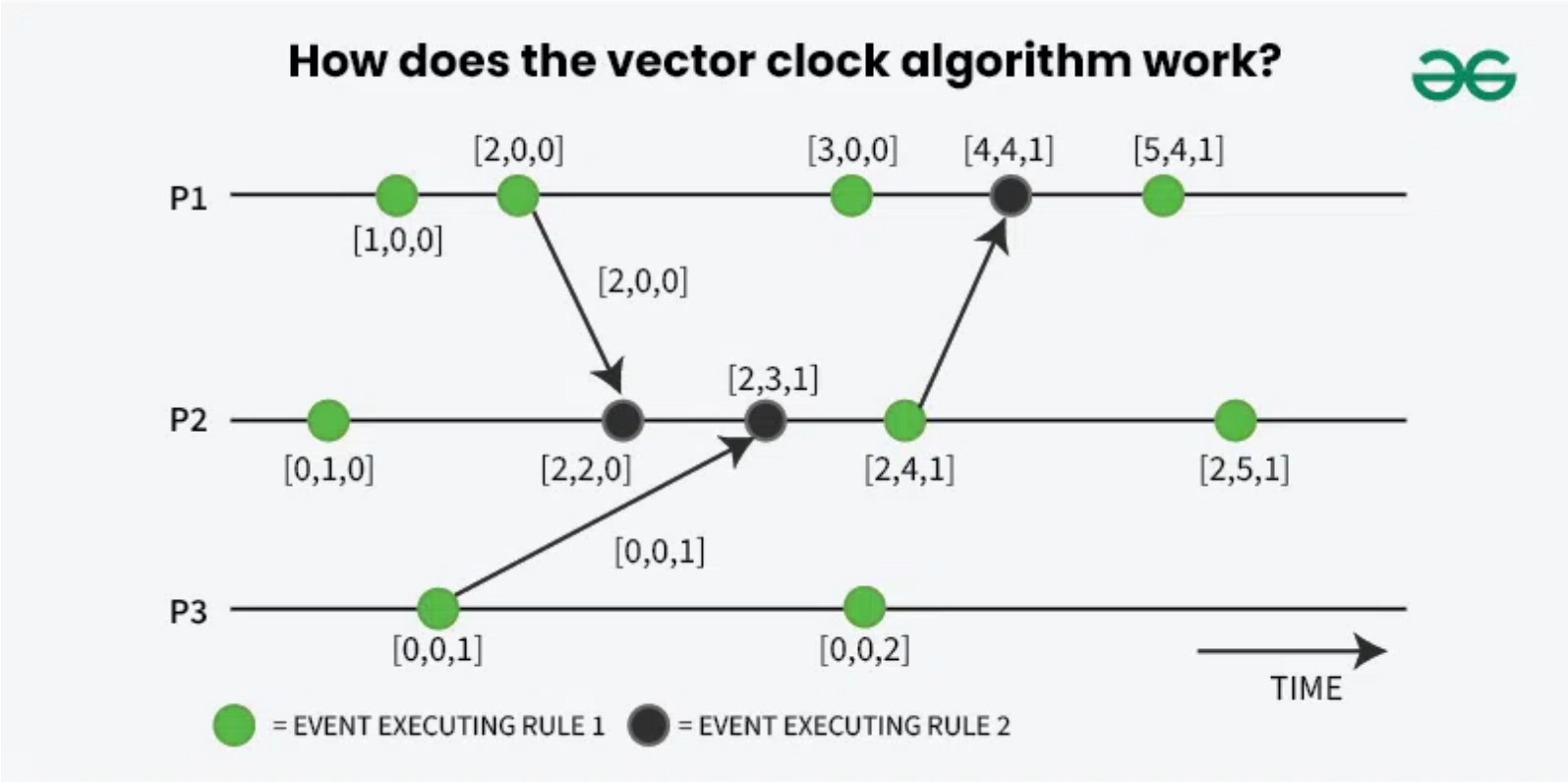


Figure 17: Vector Clock Algorithm: Each process P_i maintains a clock vector $[V_1, V_2, V_3]$. Local/send events increment its own counter (Rule 1, green), while message receipts update each element via $\max(V_i[k], V_{msg}[k])$ and increment $V_i[i]$ (Rule 2, black), enabling exact detection of causal versus concurrent write conflicts.

Source: GeeksforGeeks - Vector Clocks in Distributed Systems

1.5 Redis: High-Performance In-Memory Key-Value Architecture

1.5.1 Overview and Design Philosophy

Redis (Remote Dictionary Server), created by Salvatore Sanfilippo, is the world's most widely used in-memory key-value data structure store:

- **In-Memory Storage:** Primary data structures reside entirely in RAM, enabling sub-millisecond read/write latency ($> 100,000$ queries/second on commodity hardware).
- **Rich Data Structures:** Beyond plain strings, Redis provides native, optimized implementations of Hashes, Lists, Sets, and Sorted Sets.
- **Atomic Operations:** Every built-in command is strictly atomic, eliminating race conditions for counters, sets, and queues.
- **Persistence:** Offers durable on-disk snapshots (**RDB**) and append-only write logs (**AOF**) to protect memory state across restarts.

1.5.2 The Single-Threaded Reactor Event Loop

A frequent engineering question is: *"Why is Redis single-threaded, and why is it so fast?"*

The Three Secrets of Redis Speed

1. **No Disk I/O Bottleneck:** All reads and writes operate on memory structures in RAM.
2. **Non-Blocking I/O Multiplexing:** Redis uses an event loop based on OS primitives (`epoll` on Linux, `kqueue` on macOS). A single thread monitors thousands of client network sockets simultaneously, processing requests as packets arrive without waiting.
3. **Zero Thread Synchronization Overhead:** Single-threading eliminates:
   - CPU context switching between threads.
   - Mutex locks, spinlocks, and deadlocks.
   - Race conditions and memory cache invalidation across CPU cores.

1.5.3 Core Data Structures and Memory Expiration (TTL)

Redis extends simple key-value lookups with rich in-memory data structures:

- **The 5 Core Data Types:**
  - **String:** Plain text, serialized JSON, and atomic counters (`INCR`, `DECR`).
  - **Hash:** Key-value field pairs; ideal for user profiles and entity objects without JSON overhead.
  - **List:** Ordered collections; commonly used as FIFO worker queues (`LPUSH`, `RPOP`).
  - **Set:** Unordered unique elements; used for tags, mutual friends, and IP filtering.
  - **Sorted Set (ZSet):** Elements ordered by floating-point scores; ideal for real-time leaderboards.
- **Automatic Expiration (TTL) and Eviction:**
  - **Time-To-Live (TTL):** Keys can be assigned an expiration time (`EXPIRE key seconds`). Expired keys are automatically purged to prevent stale data.
  - **LRU Eviction:** When memory reaches its limit, Redis evicts (move out or remove) the **Least Recently Used** keys, keeping "hot" working data in RAM without crashing.

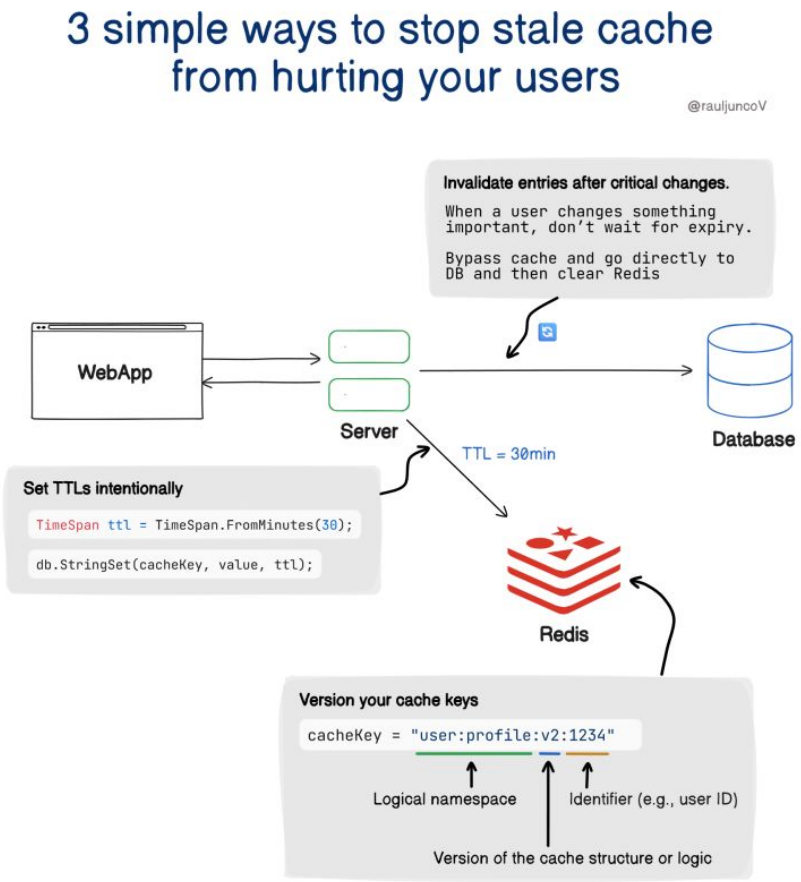


Figure 18: Best Practices for Redis Cache Expiration: Managing TTL and cache freshness requires deliberate strategies to prevent stale data. Key patterns include: (1) invalidating entries immediately upon critical database writes rather than waiting for expiry, (2) intentionally setting explicit TTL durations, and (3) versioning cache keys (e.g., `user:profile:v2:1234`) to enable seamless schema migration and clean namespace invalidation.

Source: LinkedIn - Best Practices for Redis Cache Expiration

1.5.4 Caching Architecture: The Cache-Aside Pattern (Lazy Loading)

In web applications, Redis typically serves as a fast cache in front of a slower primary database (e.g., MySQL or MongoDB):

- **1. Read Operation (Lazy Loading):**

$$\text{Client} \xrightarrow{\text{Request}} \text{Application} \xrightarrow{\text{Check Cache}} \text{Redis}$$

  - **Cache Hit:**
    If the key exists in Redis, return the cached value immediately ($<$ 1ms latency). The primary database is not queried.
  - **Cache Miss:**
    If the key does not exist, query the primary database, store the result in Redis with a TTL (e.g., 60s), and return the data to the client.

- **2. Write Operation (Cache Invalidation):**
  - When data is updated or deleted in the primary database, the application explicitly removes the cached key from Redis (`DEL key`).
  - **Note:** Deleting the key (**Delete-on-Write**) forces the next read to fetch fresh data from the database.

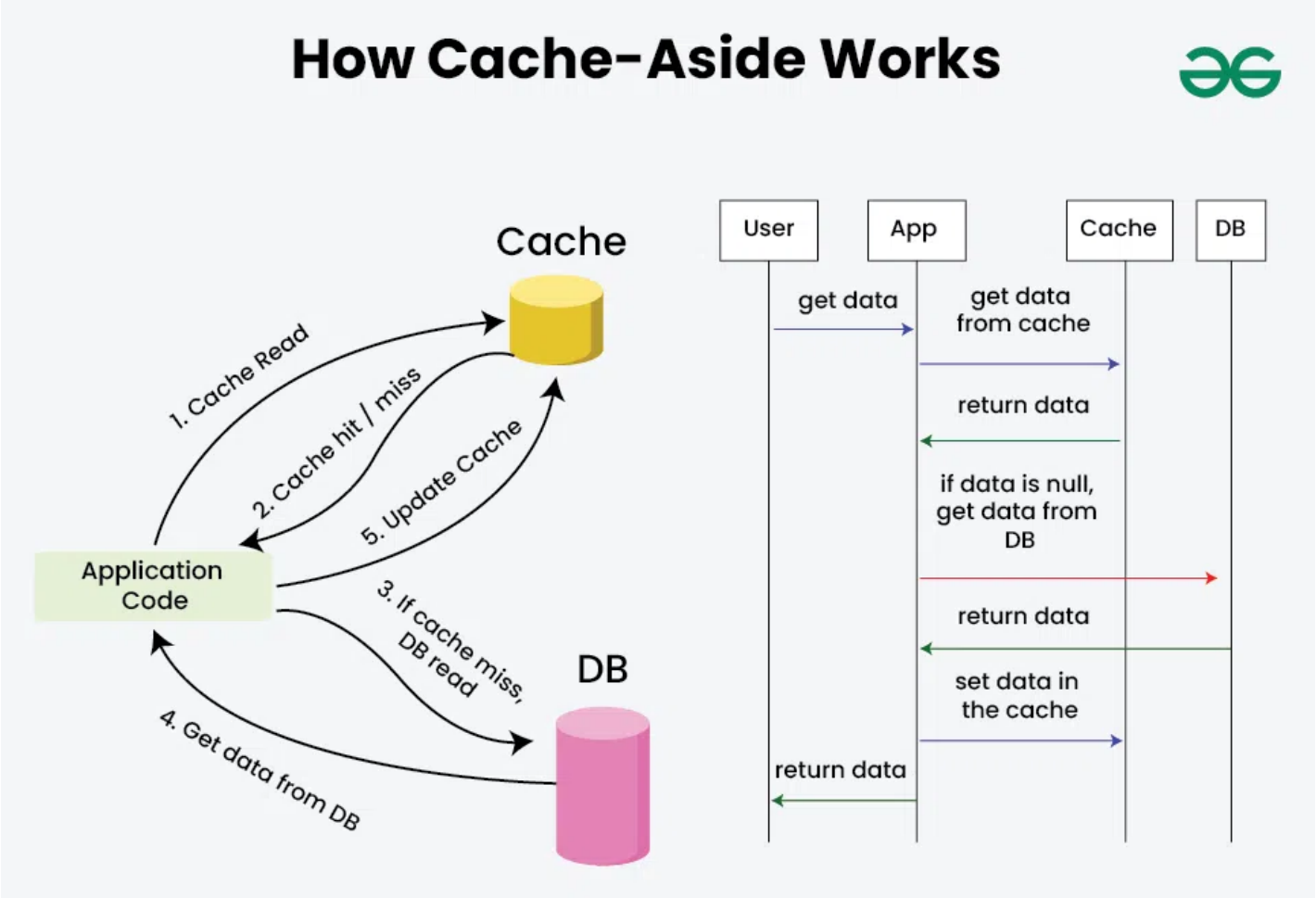


Figure 19: How Cache-Aside Works: The application coordinates data retrieval and caching. On a read request, the application first inspects the cache; if a cache miss occurs, it queries the primary database, populates the cache with the retrieved data, and returns it to the client.

Source: GeeksforGeeks - Cache-Aside Pattern

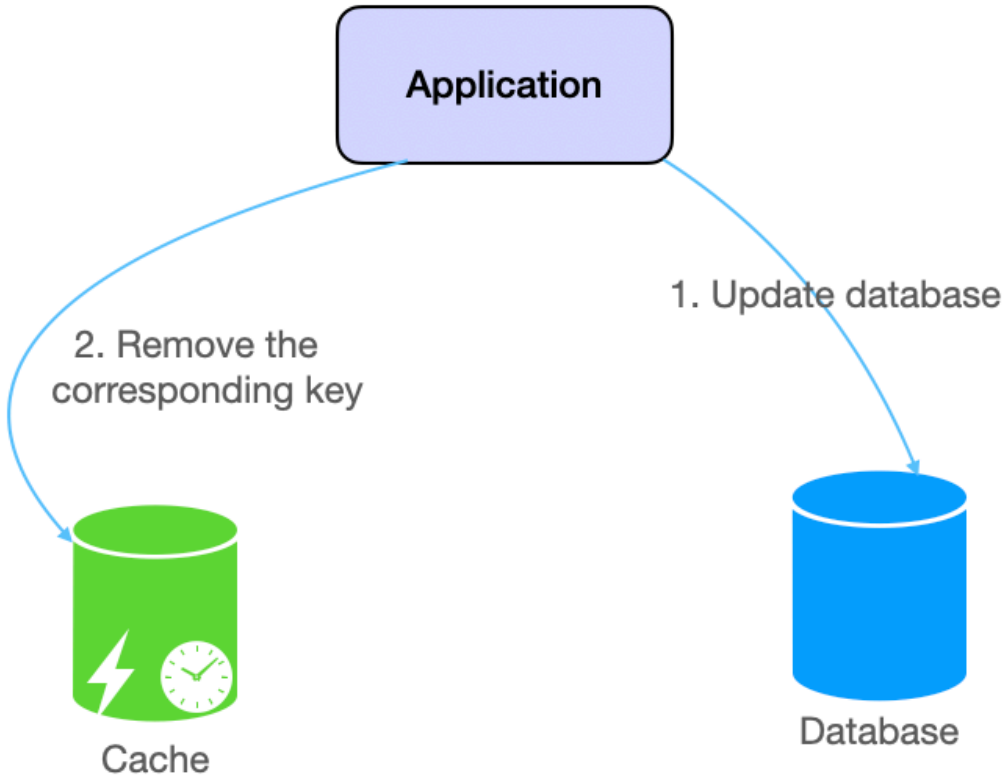


Figure 20: Cache-Aside Write Path (Cache Invalidation): When updating data, the application writes changes to the primary database and subsequently deletes the corresponding key from the cache. This Delete-on-Write strategy guarantees that stale data is purged, forcing the subsequent read request to retrieve fresh state from the database.

Source: System Design School - Cache-Aside Pattern

1.6 Redis: In-Memory Key-Value Store

Redis (Remote Dictionary Server) is an open-source, in-memory data structure store, used as a database, cache, and message broker. It supports various data structures such as strings, hashes, lists, sets, and sorted sets.

1.6.1 Basic Operations

Redis commands are atomic and easy to use.

Listing 1: Redis Basic Commands

```
# Set a key
SET user:1001 "Alice"
> OK

# Get a key
GET user:1001
> "Alice"

# Atomic Increment
SET counter 10
INCR counter
> (integer) 11

# Set Expiration (TTL) in seconds
EXPIRE user:1001 60
> (integer) 1
```

1.6.2 Transactions in Redis

Redis transactions allow the execution of a group of commands in a single step using `MULTI`, `EXEC`, `DISCARD`, and `WATCH`.

- **MULTI**: Marks the start of a transaction block. Subsequent commands are queued.
- **EXEC**: Executes all previously queued commands atomically.
- **DISCARD**: Flushes the queue and exits the transaction.

Note: Unlike relational databases, certain Redis transactions do not support rollbacks if a command fails during execution.

Listing 2: Redis Transaction Example

```
> MULTI
OK

> INCR clicks
QUEUED

> INCR hits
QUEUED

> EXEC
1) (integer) 1
2) (integer) 1
```

1.6.3 Redis with Python

To interact with Redis using Python, we use the `redis-py` library.

Installation:

```
pip install redis
```

Example Usage:

Listing 3: Redis with Python

```python
import redis

# Connect to Redis
r = redis.Redis(host='localhost', port=6379, db=0)

# Set a key
r.set('user:1001', 'Alice')

# Get a key (returns bytes)
name = r.get('user:1001')
print(name.decode('utf-8'))  # Output: Alice

# Atomic Increment
r.set('counter', 10)
r.incr('counter')
print(r.get('counter').decode('utf-8'))  # Output: 11
```

Transactions (Pipeline):

Listing 4: Redis Transaction with Python

```python
# Use a pipeline to execute multiple commands atomically
pipe = r.pipeline()

try:
    pipe.multi()
    pipe.set('account:A', 50)
    pipe.set('account:B', 150)
    pipe.execute() # Executes all commands
    print("Transaction Successful")
except redis.exceptions.RedisError as e:
    print(f"Transaction Failed: {e}")
```

Key Concept: Redis vs. MySQL Transactions

Important Distinction: In a Relational Database (like MySQL), if one command in a transaction fails, the entire transaction is **rolled back** (undone) to maintain consistency.

In **Redis**, if a command fails during execution (e.g., wrong data type), **the rest of the commands are still executed**. Redis does **not** support rollbacks. This design choice implies that Redis prioritizes performance and simplicity over the complex rollback mechanisms found in ACID databases.

1.7 Chapter Summary

- **Polyglot Persistence & NoSQL:** Polyglot persistence is an architecture that uses multiple database technologies within one system. Modern systems combine relational databases (ACID) and NoSQL databases to meet different workload requirements, such as horizontal scalability, high write throughput, and flexible schemas.
- **CAP & PACELC Theorems:** Because network partitions (P) are physically unavoidable, distributed systems must trade off Consistency (CP) vs. Availability (AP). When operating normally, they trade off Latency (L) vs. Consistency (C).
- **Consistent Hashing & Virtual Nodes:** Maps nodes and keys to a 360° ring. On cluster resize, only K/N keys move. Virtual nodes (VNodes) distribute load evenly across heterogeneous physical servers to eliminate hotspots.
- **Quorum Consensus & Conflict Resolution:** In leaderless clusters, setting $R + W > N$ guarantees strong consistency via overlapping read/write sets. Vector clocks track causal history to detect concurrent write conflicts without clock drift.
- **Redis In-Memory Architecture:** Delivers $>$ 100k ops/sec using in-memory storage, non-blocking I/O multiplexing (Reactor pattern), and a single-threaded execution core that eliminates lock contention and context switching.
- **Production Cache-Aside Strategy:** Web applications cache frequently read data using lazy loading (Cache Miss $\rightarrow$ read DB $\rightarrow$ write cache with TTL). Stale data is prevented using **Delete-on-Write** cache invalidation and structured key versioning.

1.8 Activity/Homework 1: Key-Value Stores with Python and Redis

1.8.1 Laboratory 1: Core Setup, Strings, Hashes, and Sets

Install `redis-py` via `pip install redis`. The code below initializes the connection and demonstrates core types:

Listing 5: Redis Connection, Strings, Hashes, and Sets in Python

```python
import redis

# 1. Connect (decode_responses=True returns str instead of bytes)
r = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)
print("Connected:", r.ping()) # True

# 2. Strings: Key-Value Storage & Atomic Counters
r.set("page:views", "100")
r.incr("page:views") # Atomic increment -> 101
print(f"Page views: {r.get('page:views')}")

# 3. Hashes: Structured Object Fields (No JSON serialize overhead)
r.hset("user:101", mapping={"name": "Alice", "role": "admin"})
r.hincrby("user:101", "login_count", 1) # Atomic field increment -> 1
print(f"User Object: {r.hgetall('user:101')}")

# 4. Sets: Unique Collections & Fast Intersections
r.sadd("user:101:skills", "Python", "SQL", "Redis")
r.sadd("user:102:skills", "Go", "Docker", "Redis")
print("Shared:", r.sinter("user:101:skills", "user:102:skills")) # {'Redis'}
print("Has Python?", r.sismember("user:101:skills", "Python")) # True
```

1.8.2 Laboratory 1 (Continued): Queues, Leaderboards, and TTL Expiration

Listing 6: Lists, Sorted Sets, and Key Expiration (TTL) in Python

```
# 5. Lists: FIFO Producer-Consumer Task Queue
r.delete("task_queue")
r.lpush("task_queue", "send_welcome_email")
r.lpush("task_queue", "generate_pdf_report")
task = r.rpop("task_queue") # Consumer pops the oldest task (FIFO)
print(f"Processed Task: {task}") # 'send_welcome_email'

# 6. Sorted Sets (ZSet): Real-Time Gaming Leaderboard with Dynamic Scores
r.delete("game:highscores")
r.zadd("game:highscores", {"Alice": 1450, "Bob": 2200, "Charlie": 1850})
r.zincrby("game:highscores", 500, "Alice") # Score increments dynamically to 1950

# Fetch Top 3 players in descending order with scores
top_players = r.zrevrange("game:highscores", 0, 2, withscores=True)
print("Top Players:", [(p, int(s)) for p, s in top_players])

# 7. Automatic Expiration (TTL) and Key Eviction
r.set("session:token_99", "user_auth_token", ex=30) # Expires in 30 seconds
print(f"Remaining TTL: {r.ttl('session:token_99')} seconds")
# When TTL reaches 0, Redis automatically purges the key (r.get returns None)
```

1.8.3 Laboratory 2: Practical Cache-Aside Caching in Python

In application backends, Redis acts as a fast cache in front of a persistent database (Cache-Aside pattern):

Listing 7: Standalone Cache-Aside Pattern in Python

```
import redis, json, time
r = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)

# Simulated primary database query (e.g., MySQL / MongoDB)
def query_primary_db(user_id):
    time.sleep(0.2) # Simulated disk I/O latency
    return {"user_id": user_id, "name": "Piya", "role": "Instructor"}

# 1. READ: Cache-Aside (Lazy Loading)
def get_user_profile(user_id):
    cache_key = f"cache:user:{user_id}"
    cached = r.get(cache_key)
    if cached:
        print(f"[CACHE HIT] Retrieved '{cache_key}' from Redis (<1ms)")
        return json.loads(cached)
    print(f"[CACHE MISS] '{cache_key}' absent. Fetching from DB...")
    data = query_primary_db(user_id)
    r.setex(cache_key, 60, json.dumps(data)) # Store in Redis with 60s TTL
    return data

# 2. WRITE: Cache Invalidation (Delete-on-Write)
def update_user_profile(user_id, updated_fields):
    # primary_db.update(user_id, updated_fields) # 1. Update primary DB
    r.delete(f"cache:user:{user_id}")            # 2. Evict stale cache key
    print(f"[CACHE INVALIDATED] Deleted cache:user:{user_id}")
```

1.9 Activity/Homework 2: Multi-Database Web Application with Redis Caching

1.9.1 Overview

In this activity, you will build a backend for an **Online Store Platform** that integrates three different database systems to leverage their specific strengths:

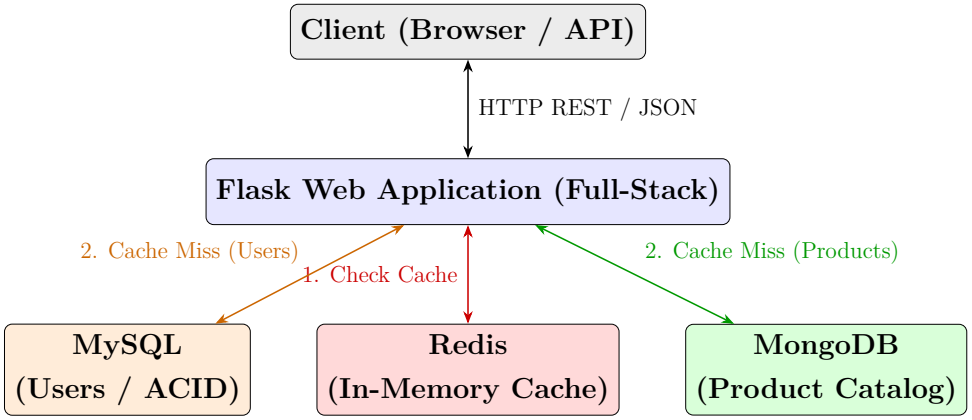
- **MySQL:** Stores structured user profiles (Relational, ACID compliance for identity and roles).
- **MongoDB:** Stores flexible product catalogs (Document-based, ideal for varying product details).
- **Redis:** Provides a high-speed caching layer for frequently accessed profiles and catalogs (In-memory, Key-Value).

1.9.2 Learning Objectives

By completing this activity, you will be able to:

- Integrate multiple database paradigms (**MySQL**, **MongoDB**, and **Redis**) into a unified Flask application.
- Implement the **Cache-Aside (Lazy Loading)** pattern for collection lists and individual entities.
- Enforce **Cache Invalidation** using the **Delete-on-Write** pattern during record creation and updates.
- Construct cache administrative endpoints to inspect memory status and audit active keys.

1.9.3 System Architecture



1.9.4 Database Design

1. MySQL: `users` Table

Designed for fixed schema and strong consistency.

- `id`: INT (PK, Auto Increment)
- `username`: VARCHAR(100)
- `email`: VARCHAR(255)
- `role`: ENUM('admin', 'user')
- `created_at`: TIMESTAMP

2. MongoDB: `products` Collection

Designed for flexibility; different products can have different attributes.

```
{
  "_id": "P1001",
  "name": "Gaming Mouse",
  "category": "Electronics",
  "price": 1299,
}
```

3. Redis: Cache Keys

- `user_list`: JSON string of all users from MySQL.
- `user:<id>`: JSON string of a specific user profile.
- `product_list`: JSON string of all products from MongoDB.
- `product:<id>`: JSON string of specific product details.

1.9.5 Activity Tasks

Activity 1: Setup MySQL Connection

Implement the `get_mysql_connection()` function in `db.py`. Use the `mysql.connector` library to connect to the local database using the following credentials:
`host=localhost`, `user=root`, `password=rootpassword`, and `database=user_db`.

Activity 2: Setup MongoDB Connection

Implement the `get_mongo_collection()` function in `db.py`. Use the `pymongo` library to connect to `mongodb://localhost:27017` and return the `products` collection from the `product_db` database.

Activity 3: Setup Redis Client

Implement the `get_redis_client()` function in `db.py`. Create a Redis client connecting to `localhost:6379` with `decode_responses=True`.

Activity 4: Implement Lazy Loading Pattern (List Retrieval)

Modify the `list_products` and `list_users` endpoints to implement the Lazy Loading pattern:

1. Check Redis for the existing list (`product_list` or `user_list`).
2. If found (**Cache Hit**), return the data immediately.
3. If not found (**Cache Miss**), fetch from the primary database and populate Redis for future requests.

Activity 5: Implement Cache Invalidation (Resource Creation)

Ensure that when a new User or Product is created via `POST` requests, the corresponding list cache in Redis is deleted. This ensures the dashboard stays up-to-date with the latest data.

Activity 6: Individual Entity Caching

Implement caching for individual lookups (`GET /users/<id>` and `GET /products/<id>`). Use key formats like `user:1` and `product:P1001`. This demonstrates more granular caching compared to full list retrieval.

Activity 7: Bulk Cache Invalidation (Management)

Implement the `/cache/remove` endpoint logic. This function should iterate through Redis and delete all relevant keys (lists and individual profiles) to allow for testing the "Cache Miss" scenario manually.

Activity 8: Cache Status and Auditing

Implement the `/cache/status` logic to return the total count of keys in Redis. This data is displayed in the "Keys" badge on the frontend dashboard.

1.9.6 Deliverables

Submit your commented `db.py` and `app.py`, a screenshot of your **Cache Explorer** with active data, and a copy of your **terminal logs** showing a sequence of Cache Misses followed by Cache Hits.